Implementation and experimentation with Dynamic Staging

LP2, 2nd May 2026

CHING Long Tin, YEUNG Sin Chun

Motivation 1
Compile-Time vs. Run-Time Work 2
Multi-Stage Programming 4
Class specialization 7
Overview 9
Implementation 13
Testing 36
Benchmarking 37

Compile-Time vs. Run-Time Work

Programmers often write code in a clear, abstract style, even when parts of it could be computed during compilation.

Original program

```mls
fun dot(xs, ys) =
  if xs is
    Nil then 0
    Cons(x, xt) and ys is
      Cons(y, yt) then
        x * y + dot(xt, yt)

fun dotWith3(v) =
  dot([1, 0, 2], v)
```

Residual program (after specialising on `[1, 0, 2]`)

```mls
fun dotWith3(v) = v(0)
+ 2 * v(2)
```

Compile-Time vs. Run-Time Work

The recursion, the pattern matches, the multiplication by `0` — all known at compile time. The residual program contains only the work that genuinely depends on the runtime input `v`.

Goal: let the programmer keep the abstract version, and have the compiler peel away the static layer automatically.

Multi-Stage Programming

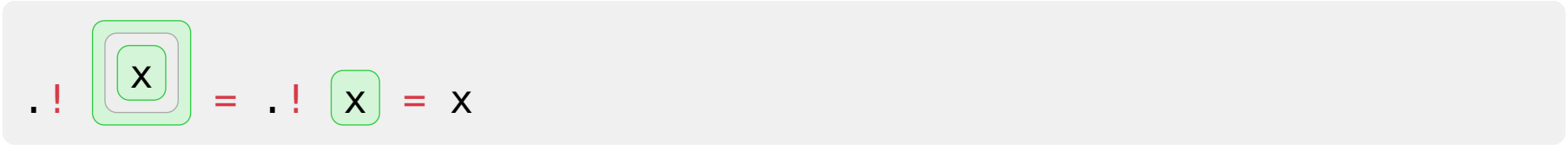
Well-known optimization technique

```
1  fun power(n, x) = if n is
2    0 then .<1>.
3    else .<.~x * .~(power(n - 1, x))>.
4  fun power2 = .! .<(x => .~(power(2, .<x>.)))>.
```

In here, green to annotates code to be executed in the next stage, and grey executed in the current stage.

Multi-Stage Programming

Rule for `.!`: evaluate until we the whole code fragment is in the next stage, then move it to the current stage.



During evaluation, we able to pre-compute certain parts of the code, reducing runtime.

Multi-Stage Programming

```
1  x => power(2, x)                                                    mls
2  x => if 2 is 0 then 1 else x * power(2 - 1, x)
3  x => x * power(1, x)
4  x => x * if 1 is 0 then 1 else x * power(1 - 1, x)
5  x => x * x * if 0 is 0 then 1 else x * power(0 - 1, x)
6  x => x * x * 1
7  fun power2 = x => x * x * 1
```

Class specialization

```
1  class B(val x) with
2    fun f() = ...
3  // ...
4  if x is
5    1 then new D1(1)
6    2 then new D2(2)
7    3 then new D3(3)
8
9  x.f()
```

```
…                    …
8   if x is                    mls
9     D1 then x.D1_f1()
10    D2 then x.D2_f1()
11    D3 then x.D3_f1()
```

Even if `x` has a known class shape, we cannot remove matching arms.

Specialization is done through typing, so we know that `x : B`, but we do not know which specific derived class of `B` is used.

Class specialization

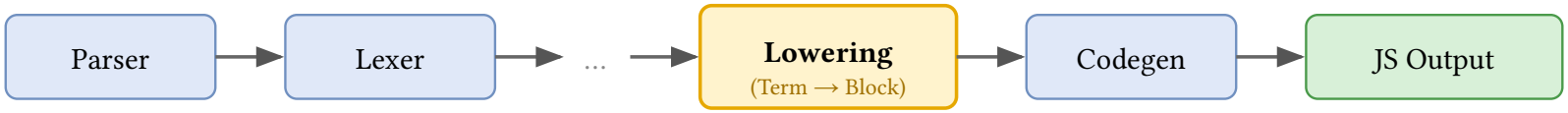
```mls
data class Foo(x, y) extends Bar(x+1, y+1)

if new Foo(1, 2) is
  Bar(x, y) then ...
```

Under single inheritance, matching arms runs into problems.

```mls
class Foo$1$2 extends Bar
class Bar$2$3
```

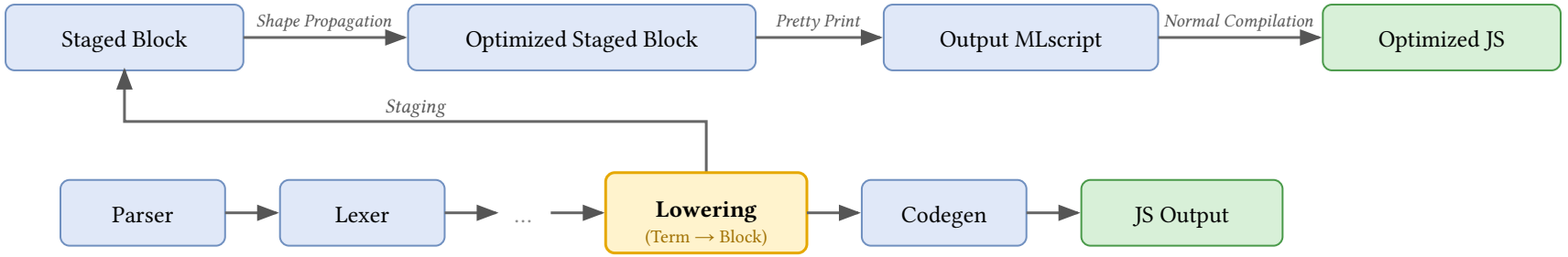
Motivation 1
Overview 9
MLscript Compiler 10
Lowering Pass 11
instrumentation 12
Shape 12
Implementation 13
Testing 36
Benchmarking 37

MLscript Compiler



Lowering Pass

We do instrumentation from Scala Block to Scala Block.



Shape

We focus on tracking the shape of the values defined in Block, paths and results.

$$s ::= \iota \mid \mathbf{dyn} \mid [\overline{s}] \mid C(\overline{n : s}) \mid \bot \mid s \cup s$$

Motivation 1
Overview 9
Implementation 13
What's this for? 14
Staging 15
Shape Propagation 20
Printing Staged Block 34
Testing 36
Benchmarking 37

What's this for?

```mls
staged module A with
  fun pow(@dynamic x, n) = if n is
    0 then 1
    else x * pow(x, n-1)
  fun sq(x) = pow(x, 2)
```

```
…                                   …
  fun sq(x) = x * x * 1
```
mls

This example touches on shape tracking, match arm removal, and nested specialization.

Staging

```scala
case class Match(scrut, arms, dflt, rest) extends Block
case class Return(scrut, arms, dflt, rest) extends Block
case class Assign(lhs, rhs, rest) extends Block
```

```mls
class Block with
  constructor
    Return(res, implct)
    Match(scrut, arms, dflt, rest)
    Assign(lhs, rhs, rest)
```

For any Scala Block data, we can recreate the same structure with **Staged Block**.

Staging

```
fun pow(x, n) = if n is 0 then 1 else x * pow(x, n-1)
```

```Scala
val pow = Symbol("pow"); val x = Symbol("x"); val n = Symbol("n")
val t1 = Symbol("tmp"); val t2 = Symbol("tmp")
val sub = Symbol("-"); val mul = Symbol("*")
FunDefn(pow, [x, n], Scoped(HashSet(t1, t2), Match(Ref(n),
  [[ Lit(0), Return(Lit(1)) ]],
  Assign(t1, Call(sub, [n, Lit(1)]),
    Assign(t2, Call(pow, [x, t1]),
      Return(Call(mul, [x, t2]))
    )
  ), End())
))
```

As long as we have the corresponding constructor, we can copy over the structure to the Staged Block.

Staging

```
fun pow(x, n) = if n is 0 then 1 else x * pow(x, n-1)
```

```mls
let pow = Symbol("pow"); let x = Symbol("x"); let n = Symbol("n")
let t1 = Symbol("tmp"); let t2 = Symbol("tmp")
let sub = Symbol("-"); let mul = Symbol("*")
FunDefn(pow, [x, n], Scoped([t1, t2], Match(Ref(n),
  [[ Lit(0), Return(Lit(1)) ]],
  Assign(t1, Call(sub, [n, Lit(1)]),
    Assign(t2, Call(pow, [x, t1]),
      Return(Call(mul, [x, t2]))
    )
  ), End())
))
```

As long as we have the corresponding constructor, we can copy over the structure to the Staged Block.

Staging

We perform structural induction to stage each type of Scala Block.

`Value.Lit(lit)`

```mls
ValueLit(lit)
```

Example: `1` $\mapsto$ `ValueLit(1)`

`Symbol(name)`

```mls
Symbol(name)
```

Example:

- `x` $\mapsto$ `Symbol("x")`
- `c` $\mapsto$ ...?

This is not enough for staging symbols. We'll revisit this case later on.

Staging

We perform structural induction to stage each type of Scala Block.

`Value.Lit(lit)`

```mls
1  ValueLit(lit)
```

Example: `1` ↦ `ValueLit(1)`

`Value.Ref(sym)`

```mls
1  let l = sym
2  ValueRef(l)
```

Example: `x` ↦ `ValueRef(Symbol("x"))`

`Select(qual, name)`

```mls
1  let q = qual
2  let n = name
3  Select(q, n)
```

Example: `x.p` ↦ `Select(ValueRef(Symbol("x")), Symbol("p"))`

Staging

```
Tuple([x0, x1, ...])
```

```
1  let s0 = x0                                                    mls
2  let s1 = x1
…                                                                 …
4  Tuple([x0, x1, ...])
```

The other Scala Block cases are similar, staging the parameters of the Scala Block by induction and recreating the corresponding Staged Block counterpart.

Staging

Staging Symbols

- We need more information about the Symbol in the original stage (particularly, tracking the previous stage values)

  Add redirection within current stage to allow access for next module

```mls
staged class A with
  fun f() = M.f()
  val redirect_M = M
```

Staging

Staging Symbols

- Uniqueness of Symbols

  We want Symbols for the same object in the previous stage to be unique in the current stage across functions and compilations

  For local functions, we can maintain a map during staging to reuse a staged symbol.

  For class and module symbols, we need to cache and use first instance of a symbol within the staged code.

Shape Propagation

A **Shape** captures what we statically know about a value at compile time.

$$s ::= \iota \mid \mathbf{dyn} \mid [\overline{s}] \mid C(\overline{n : s}) \mid \bot \mid s \cup s$$

We write $[\![\texttt{s}]\!]$ to denote the shape of an expression, distinguishing it from actual values.

```mls
fun f(p, cond) =
  let a = 42                          // 42
  let b = C(p, 2)                     // C(dyn, 2)
  let c = if cond
    then [1, 2, 3] else C(1,2)        // {[1,2,3], C(1,2)}
  let d = if b is C then 0 else 1 // 0
```

Shapes are tracked in a context Γ mapping paths to their **ShapeSet**.

Shape Propagation

Before the formal rules, let's trace propagation on this module:

```mls
class C(val n)
staged module If2 with
  fun f(x) =
    let y
    if x is C then  y = x.n
    else y = 0
    y + 1
  fun test()     = f(C(2))
  fun test2(dyn) = f(C(dyn))
  fun test3()    = f(0)
```

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\longmapsto [\![$`C(n: 2)`$]\!]$.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` $\longmapsto [\![$`C(n: 2)`$]\!]$

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\longmapsto [\![$`C(n: 2)`$]\!]$.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` $\longmapsto [\![$`C(n: 2)`$]\!]$

`y` $\longmapsto [\![\bot]\!]$

1. `let y`: extend ctx with `y` $\longmapsto [\![\bot]\!]$

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\longmapsto [\![$`C(n: 2)`$]\!]$.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` $\longmapsto [\![$`C(n: 2)`$]\!]$

`y` $\longmapsto [\![\bot]\!]$

1. `let y`: extend ctx with `y` $\longmapsto [\![\bot]\!]$
2. `if x is C`: `filter(`$[\![$`C(n: 2)`$]\!]$`, C)` = $[\![$`C(n: 2)`$]\!]$ so `then` is viable; `rest` = $[\![\bot]\!]$ so `else` is dead

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\longmapsto [\![$`C(n: 2)`$]\!]$.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` $\longmapsto [\![$`C(n: 2)`$]\!]$

`y` $\longmapsto [\![$`2`$]\!]$

1. `let y`: extend ctx with `y` $\longmapsto [\![\bot]\!]$
2. `if x is C`: `filter(`$[\![$`C(n: 2)`$]\!]$`, C)` = $[\![$`C(n: 2)`$]\!]$ so `then` is viable; `rest` = $[\![\bot]\!]$ so `else` is dead
3. In `then`: `sop(x.n)` = `sel(`$[\![$`C(n: 2)`$]\!]$`, n)` = $[\![$`2`$]\!]$, so `y` $\longmapsto [\![$`2`$]\!]$

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` ⟼ ⟦`C(n: 2)`⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦`C(n: 2)`⟧

`y` ⟼ ⟦`2`⟧

1. `let y`: extend ctx with `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦`C(n: 2)`⟧`, C)` = ⟦`C(n: 2)`⟧ so `then` is viable; `rest` = ⟦⊥⟧ so `else` is dead
3. In `then`: `sop(x.n)` = `sel(`⟦`C(n: 2)`⟧`, n)` = ⟦`2`⟧, so `y` ⟼ ⟦`2`⟧
4. `y + 1` folds to ⟦`3`⟧, then `Return 3`

Cached as `f_C_Lit2`; body folds entirely to the constant `3`.

Shape Propagation

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: ⟦`C(n: dyn)`⟧.

Specialise `f` with `x` ⟼ ⟦`C(n: dyn)`⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦`C(n: dyn)`⟧

Shape Propagation

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: ⟦`C(n: dyn)`⟧.

Specialise `f` with `x` ⟼ ⟦`C(n: dyn)`⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦`C(n: dyn)`⟧

`y` ⟼ ⟦⊥⟧

1. `let y: y` ⟼ ⟦⊥⟧

Shape Propagation

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: ⟦`C(n: dyn)`⟧.

Specialise `f` with `x` ⟼ ⟦`C(n: dyn)`⟧.

```
1  fun f(x) =                                  mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦`C(n: dyn)`⟧

`y` ⟼ ⟦⊥⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦`C(n: dyn)`⟧`, C)` = ⟦`C(n: dyn)`⟧ so `then` is viable; `rest` = ⟦⊥⟧ so `else` is dead

Shape Propagation

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: ⟦`C(n: dyn)`⟧.

Specialise `f` with `x` ⟼ ⟦`C(n: dyn)`⟧.

```
1  fun f(x) =                                  mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦`C(n: dyn)`⟧

`y` ⟼ ⟦`dyn`⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦`C(n: dyn)`⟧`, C)` = ⟦`C(n: dyn)`⟧ so `then` is viable; `rest` = ⟦⊥⟧ so `else` is dead
3. In `then`: `sop(x.n)` = `sel(`⟦`C(n: dyn)`⟧`, n)` = ⟦`dyn`⟧

Shape Propagation

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: ⟦`C(n: dyn)`⟧.

Specialise `f` with `x` ⟼ ⟦`C(n: dyn)`⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦`C(n: dyn)`⟧

`y` ⟼ ⟦`dyn`⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦`C(n: dyn)`⟧`, C)` = ⟦`C(n: dyn)`⟧ so `then` is viable; `rest` = ⟦⊥⟧ so `else` is dead
3. In `then`: `sop(x.n)` = `sel(`⟦`C(n: dyn)`⟧`, n)` = ⟦`dyn`⟧
4. `y + 1`: cannot fold (⟦`dyn`⟧ `+ 1`); emit code, result ⟦`dyn`⟧

Cached as `f_C_Dyn`; body keeps the `y = x.n` assignment.

Shape Propagation

Trace 3: `test3() = f(0)`

Argument shape: ⟦0⟧ (literal).

Specialise `f` with x ⟼ ⟦0⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

x ⟼ ⟦0⟧

Shape Propagation

Trace 3: `test3() = f(0)`

Argument shape: ⟦0⟧ (literal).

Specialise `f` with `x` ⟼ ⟦0⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦0⟧

`y` ⟼ ⟦⊥⟧

1. `let y: y` ⟼ ⟦⊥⟧

Shape Propagation

Trace 3: `test3() = f(0)`

Argument shape: ⟦0⟧ (literal).

Specialise `f` with `x` ⟼ ⟦0⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦0⟧

`y` ⟼ ⟦⊥⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦0⟧`, C)` = ⟦⊥⟧ so `then` is dead; `rest(`⟦0⟧`, C)` = ⟦0⟧ so `else` is viable

Shape Propagation

Trace 3: `test3() = f(0)`

Argument shape: ⟦0⟧ (literal).
Specialise `f` with `x` ⟼ ⟦0⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦0⟧

`y` ⟼ ⟦0⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦0⟧`, C)` = ⟦⊥⟧ so `then` is dead; `rest(`⟦0⟧`, C)` = ⟦0⟧ so `else` is viable
3. In `else`: `y` ⟼ ⟦0⟧

Shape Propagation

Trace 3: `test3() = f(0)`

Argument shape: ⟦0⟧ (literal).
Specialise `f` with `x` ⟼ ⟦0⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx
`x` ⟼ ⟦0⟧
`y` ⟼ ⟦0⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦0⟧`, C)` = ⟦⊥⟧ so `then` is dead; `rest(`⟦0⟧`, C)` = ⟦0⟧ so `else` is viable
3. In `else`: `y` ⟼ ⟦0⟧
4. `y + 1` folds to ⟦1⟧, then `Return 1`

Cached as `f_Lit0`.

Shape Propagation

Resulting Cache

After all three calls, the staged module contains:

```mls
module If2 with
  fun f_C_Dyn(x) =
    let y
    y = x.n
    y + 1                    // dyn case: code is kept
  fun f_C_Lit2(x) = 3        // fully specialised
  fun f_Lit0()    = 1        // fully specialised
  fun test()      = 3
  fun test2(dyn)  = f_C_Dyn(C$If2(dyn))
  fun test3()     = 1
```

Shape Propagation

Dynamic Dispatching

When a method is called on a value with a **union shape**, we split by class and dispatch separately.

```
1  staged class B1(val y) with  fun call(x) = x + 2 + y
2  staged class B2(val y) with  fun call(x) = x + y
3  staged module M with
4    fun twice(f, x) = f.call(f.call(x))
5    fun pick(x, y, b) = if b then x else y
6    fun f(b) =
7      let m = pick(new B1(2), new B2(3), b)
8      twice(m, 5)
```

Shape Propagation

Trace A: `f(dyn)`

Specialise `f` with `b` $\longmapsto [\![\texttt{dyn}]\!]$.

```
1  fun f(b) =                          mls
2    let m = pick(new B1(2),
     new B2(3), b)
3    twice(m, 5)
```

ctx

`b` $\longmapsto [\![\texttt{dyn}]\!]$

Shape Propagation

Trace A: `f(dyn)`

Specialise `f` with `b` $\longmapsto [\![\texttt{dyn}]\!]$.

```
1  fun f(b) =                                  mls
2      let m = pick(new B1(2), new B2(3), b)
3      twice(m, 5)
```

ctx

`b` $\longmapsto [\![\texttt{dyn}]\!]$

`m` $\longmapsto [\![\bot]\!]$

1. `let m`: extend ctx with `m` $\longmapsto [\![\bot]\!]$

Shape Propagation

Trace A: `f(dyn)`

Specialise `f` with `b` $\mapsto [\![\texttt{dyn}]\!]$.

```
1  fun f(b) =                                  mls
2      let m = pick(new B1(2), new B2(3), b)
3      twice(m, 5)
```

ctx

`b` $\mapsto [\![\texttt{dyn}]\!]$

`m` $\mapsto [\![\bot]\!]$

1. `let m`: extend ctx with `m` $\mapsto [\![\bot]\!]$
2. Call `pick(new B1(2), new B2(3), b)`: argument shapes are $[\![\texttt{B1(2)}]\!]$, $[\![\texttt{B2(3)}]\!]$, $[\![\texttt{dyn}]\!]$. Recurse into `pick` with a fresh ctx
   $\rightarrow$ **go to Trace B**

Shape Propagation

Trace B: `pick(B1(2), B2(3), dyn)`

```
1  fun pick(x, y, b) =        mls
2    if b then x
3    else y
```

ctx

fresh ctx for `pick`

$\texttt{x} \longmapsto [\![\texttt{B1(2)}]\!]$

$\texttt{y} \longmapsto [\![\texttt{B2(3)}]\!]$

$\texttt{b} \longmapsto [\![\texttt{dyn}]\!]$

Shape Propagation

Trace B: `pick(B1(2), B2(3), dyn)`

```
1  fun pick(x, y, b) =        mls
2    if b then x
3    else y
```

ctx

fresh ctx for `pick`

`x` ⟼ ⟦`B1(2)`⟧

`y` ⟼ ⟦`B2(3)`⟧

`b` ⟼ ⟦`dyn`⟧

1. `if b`: `b` is ⟦`dyn`⟧, so both branches viable

Shape Propagation

Trace B: `pick(B1(2), B2(3), dyn)`

```
1  fun pick(x, y, b) =        mls
2    if b then x
3    else y
```

ctx

fresh ctx for `pick`

`x` ⟼ ⟦`B1(2)`⟧

`y` ⟼ ⟦`B2(3)`⟧

`b` ⟼ ⟦`dyn`⟧

1. `if b`: `b` is ⟦`dyn`⟧, so both branches viable
2. `then` arm returns `x`, shape = ⟦`B1(2)`⟧

Shape Propagation

Trace B: `pick(B1(2), B2(3), dyn)`

```
1  fun pick(x, y, b) =        mls
2    if b then x
3    else y
```

ctx

fresh ctx for `pick`

`x` ⟼ ⟦`B1(2)`⟧

`y` ⟼ ⟦`B2(3)`⟧

`b` ⟼ ⟦`dyn`⟧

1. `if b`: `b` is ⟦`dyn`⟧, so both branches viable
2. `then` arm returns `x`, shape = ⟦`B1(2)`⟧
3. `else` arm returns `y`, shape = ⟦`B2(3)`⟧

Shape Propagation

Trace B: `pick(B1(2), B2(3), dyn)`

```
1  fun pick(x, y, b) =        mls
2    if b then x
3    else y
```

ctx

fresh ctx for `pick`

`x` ⟼ ⟦`B1(2)`⟧

`y` ⟼ ⟦`B2(3)`⟧

`b` ⟼ ⟦`dyn`⟧

1. `if b`: `b` is ⟦`dyn`⟧, so both branches viable
2. `then` arm returns `x`, shape = ⟦`B1(2)`⟧
3. `else` arm returns `y`, shape = ⟦`B2(3)`⟧
4. Result shape: ⟦`B1(2)` ∪ `B2(3)`⟧. Cached as `pick1`. Return to Trace A.

Shape Propagation

Trace A (continued): back in `f`

```
1  fun f(b) =                                  mls
2      let m = pick(new B1(2), new B2(3), b)
3      twice(m, 5)
```

ctx

$\texttt{b} \longmapsto [\![\texttt{dyn}]\!]$

$\texttt{m} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

Shape Propagation

Trace A (continued): back in `f`

```
1  fun f(b) =                                  mls
2      let m = pick(new B1(2), new B2(3), b)
3      twice(m, 5)
```

ctx

$\texttt{b} \longmapsto [\![\texttt{dyn}]\!]$

$\texttt{m} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

1. Bind `m` to `pick`'s returned shape = $[\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

Shape Propagation

Trace A (continued): back in `f`

```
1  fun f(b) =                          mls
2      let m = pick(new B1(2),
       new B2(3), b)
3      twice(m, 5)
```

ctx

$\texttt{b} \longmapsto [\![\texttt{dyn}]\!]$

$\texttt{m} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

1. Bind `m` to `pick`'s returned shape = $[\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$
2. `twice(m, 5)`: `m` has **union shape** → **go to Trace C** and specialise as `twice1`

Shape Propagation

Trace C: `twice({B1(2), B2(3)}, 5)`

```mls
fun twice(f, x) =
  let tmp
  tmp = f.call(x)
  f.call(tmp)
```

ctx

$f \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

$x \longmapsto [\![5]\!]$

Recall:

```mls
class B1(y) with
  fun call(x) = x + 2 + y
class B2(y) with
  fun call(x) = x + y
```

Shape Propagation

Trace C: `twice({B1(2), B2(3)}, 5)`

```mls
1  fun twice(f, x) =
2    let tmp
3    tmp = f.call(x)
4    f.call(tmp)
```

ctx

$\texttt{f} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

$\texttt{x} \longmapsto [\![\texttt{5}]\!]$

$\texttt{tmp} \longmapsto [\![\bot]\!]$

1. `let tmp`: extend ctx with $\texttt{tmp} \longmapsto [\![\bot]\!]$

Recall:

```mls
1  class B1(y) with
2    fun call(x) = x + 2 + y
3  class B2(y) with
4    fun call(x) = x + y
```

Shape Propagation

Trace C: `twice({B1(2), B2(3)}, 5)`

```mls
1  fun twice(f, x) =
2    let tmp
3    tmp = f.call(x)
4    f.call(tmp)
```

Recall:

```mls
1  class B1(y) with
2    fun call(x) = x + 2 + y
3  class B2(y) with
4    fun call(x) = x + y
```

ctx

$\texttt{f} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

$\texttt{x} \longmapsto [\![\texttt{5}]\!]$

$\texttt{tmp} \longmapsto [\![\texttt{\{9, 8\}}]\!]$

1. `let tmp`: extend ctx with $\texttt{tmp} \longmapsto [\![\bot]\!]$
2. `f.call(x)` on union $\rightarrow$ **split by class**:
   - $\texttt{f} = [\![\texttt{B1(2)}]\!]$: cache `f.call1()` $\rightarrow [\![\texttt{9}]\!]$
   - $\texttt{f} = [\![\texttt{B2(3)}]\!]$: cache `f.call1()` $\rightarrow [\![\texttt{8}]\!]$

Shape Propagation

Trace C (continued): second call

```mls
1  fun twice(f, x) =
2    let tmp
3    tmp = f.call(x)
4    f.call(tmp)
```

Recall:

```mls
1  class B1(y) with fun call(x) = x + 2 + y
2  class B2(y) with fun call(x) = x + y
```

ctx

$\texttt{f} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

$\texttt{x} \longmapsto [\![\texttt{5}]\!]$

$\texttt{tmp} \longmapsto [\![\texttt{\{9, 8\}}]\!]$

3. `f.call(tmp)` with `tmp` in $[\![\texttt{\{9, 8\}}]\!]$ (now $[\![\texttt{dyn}]\!]$): split again, body kept as code
   - `f` = $[\![\texttt{B1}]\!]$: cache `f.call2(x) = x + 2 + 2`
   - `f` = $[\![\texttt{B2}]\!]$: cache `f.call2(x) = x + 3`

Shape Propagation

Resulting Cache

After propagation, `M` and the staged classes contain:

mls

```
1  module M with
2    fun f(b) =
3      let m, tmp1, tmp2
4      tmp1 = new B1(2)
5      tmp2 = new B2(3)
6      m = pick1(tmp1, tmp2, b)
7      twice1(m)
8    fun pick1(x, y, b) =
9      if b then new B1(2) else new
       B2(3)
```

```
1    fun twice1(f) =
2      let tmp
3      if f is
4        B1 then tmp = f.call1()
5        B2 then tmp = f.call1()
6      if f is
7        B1 then f.call2(tmp)
8        B2 then f.call2(tmp)
```

Shape Propagation

mls

```
1   class B1(y) with
2     fun call1() = 9
3     fun call2(x) =
4       let tmp
5       tmp = x + 2
6       tmp + 2
7
8   class B2(y) with
9     fun call1() = 8
10    fun call2(x) = x + 3
```

Printing Staged Block

After all the specialized functions are completed, we need to write the functions to a new file.

mls

```
staged module M with
  val funCache = new Map([
    ["f1", ...],
    ["f2", ...],
    ["g1", ...]
  ])
```

```
module M with
  fun f1(x, y) = ...
  fun f2() = ...
  fun g1() = ...
```

Printing Staged Block

This is a similar process to staging, but done in reverse.

As before, we need extra care when handling symbols.

Motivation 1
Overview 9
Implementation 13
Testing 36
Benchmarking 37

relevant? there's nothing really notable compared to ordinary mlscript development (diff/compile tests are already features within it)

Motivation 1
Overview 9
Implementation 13
Testing 36
Benchmarking 37
Model-View-Projection transformation 38

Model-View-Projection transformation

basically we're pretty good at partially evaluating matrices ^w^.

Time: about 2x (from 2s to 1s, eh...)

Space: don't worry about it